12-TONE EQUAL TEMPERAMENT: A LEAN 4 PROOF

KEITH BRAITHWAITE

Abstract. We present a Lean 4 formalisation of the theorem that 12-tone equal temperament (12TET) is the unique tuning system satisfying a small set of musically-motivated axioms. Intervals are modelled as elements of $\mathbb{Z}/m\mathbb{Z}$. The main theorem states $m = 12$; a subsidiary theorem shows the major/minor interval class has exactly 8 members.

Contents

1. Introduction 2
Model 2
The predicate isPerfect 2
Module architecture 2
2. Axioms 2
2.1. A1 and A2: the tritone 2
2.3. A4: perfect intervals are not self-inverse 3
2.4. A5: the perfect class is exactly $\{p, p^{-1}\}$ 3
2.5. A6: minimality of the perfect interval above the tritone 3
2.6. A7: five semitones above the perfect interval reaches unison 3
2.7. A8: a major/minor interval exists 4
3. Derived Facts 4
Shared consequences 4
m is even 4
$m \neq 2$ 5
$p = t + 1$, derived from A6 5
$m \mid 12$ 6
$m \neq 6$ and $m \neq 4$ 6
The 12TET theorem: $m = 12$ 7
Major/minor intervals as a least fixed point 7
Cardinality 10
4. The Partition Theorem 11
5. Risks to Validity 12
Axiom consistency 12
The numeral 5 in A7 13
The minimality axiom A6 13
The opaque predicate isPerfect 13
6. Further Work 13
Appendix A. Lean 4 Proof Tactics 14

Date: April 4, 2026.

1. Introduction

In 12-tone equal temperament (12TET) the octave is divided into 12 equal semitones. This paper asks: is there a small, musically well-motivated set of axioms that *forces* $m = 12$? We answer yes, by formalising the proof in Lean 4 with the Mathlib library.

Model. Harmonic intervals are modelled as elements of $\mathbb{Z}/m\mathbb{Z}$ for an unknown m. The four interval classes are:

Unison: the identity element, 0.
Tritone: the unique non-zero self-inverse element t ($t + t = 0$).
Perfect: a primitive class constrained by axioms; exactly the pair $\{p, p^{-1}\}$.
Major/Minor: the least fixed point of a ± 1 step operator starting from $\{p, p^{-1}\}$, stopping at $\{0, t\}$.

Inversion is additive negation: $i^{-1} = -i$. This gives $i + i^{-1} = 0$ by the ring axioms, corresponding to the musical fact that an interval and its inverse span an octave.

The predicate isPerfect. The predicate isPerfect is declared `opaque` in Lean: it has no explicit definition and can only be manipulated via the axioms. This is essential. Defining it as $i \neq 0 \wedge \neg\, \mathsf{isSelfInverse}\, i$ would make every major/minor interval "perfect", collapsing the four classes to two.

Listing 1. `Basic.lean`: the opaque perfect-interval predicate

```
opaque isPerfect [NeZero m] : HarmonicInterval m → Prop
```

Module architecture. Figure 1 shows the eight `.lean` files and their import dependencies. Each rectangle lists the private declarations inside; public declarations flow along the labelled arrows to the modules that import them.

2. Axioms

We use eight axioms. All are stated as Lean `axiom` declarations, which are trusted without proof.

2.1. A1 and A2: the tritone.

A non-zero self-inverse interval exists and is unique. A1 forces m to be even; A2 rules out $m = 8$ (where both 2 and 6 are self-inverse).

Listing 2. `Axioms.lean`: A1 and A2

```
axiom tritone_exists [NeZero m] :
    ∃ t : HarmonicInterval m, IsTritone t
axiom tritone_unique [NeZero m]
    (s t : HarmonicInterval m)
    (hs : IsTritone s)
    (ht : IsTritone t) : s = t
```

Listing 3. `Axioms.lean`: A3

```
axiom perfect_exists [NeZero m] : ∃ p : HarmonicInterval m, isPerfect p
```

2.3. **A4: perfect intervals are not self-inverse.** `perfect_nonzero` is derived immediately from A4: since unison is self-inverse, a perfect interval cannot be unison.

Listing 4. `Axioms.lean`: A4 and derived `perfect_nonzero`

```
axiom perfect_not_SI [NeZero m]
    (p : HarmonicInterval m) (hp : isPerfect p) : ¬isSelfInverse p
theorem perfect_nonzero [NeZero m]
    (p : HarmonicInterval m) (hp : isPerfect p) : p ≠ 0 :=
  fun h ⟹ perfect_not_SI p hp (h ▷ unison_is_self_inverse)
```

2.4. **A5: the perfect class is exactly $\{p, p^{-1}\}$.** `perfect_unique` and `perfect_inv_closed` are derived directly from A5 by taking the two directions of the biconditional.

Listing 5. `Axioms.lean`: A5 and derived consequences

```
axiom perfect_iff [NeZero m]
    (p : HarmonicInterval m) (hp : isPerfect p)
    (q : HarmonicInterval m) : isPerfect q ↔ q = p ∨ q = p⁻¹

theorem perfect_unique [NeZero m]
    (p q : HarmonicInterval m) (hp : isPerfect p) (hq : isPerfect q) : q = p ∨ q
    = p⁻¹ :=
  (perfect_iff p hp q).mp hq

theorem perfect_inv_closed [NeZero m]
    (p : HarmonicInterval m) (hp : isPerfect p) : isPerfect p⁻¹ :=
  (perfect_iff p hp p⁻¹).mpr (Or.inr rfl)
```

2.5. **A6: minimality of the perfect interval above the tritone.** Rather than asserting $p = t + 1$ directly (which would introduce an unexplained literal), A6 states that p is the *least* non-self-inverse element of $\mathbb{Z}/m\mathbb{Z}$ whose value exceeds that of t. The equation $p = t + 1$ is then *derived* in Section 3.

Listing 6. `Axioms.lean`: A6

```
axiom perfect_least_above_tritone [NeZero m]
    (p t : HarmonicInterval m) (hp : isPerfect p) (ht : IsTritone t) :
    ZMod.val t < ZMod.val p ∧
    ∀ i : HarmonicInterval m, ¬isSelfInverse i →
      ZMod.val t < ZMod.val i → ZMod.val p ≤ ZMod.val i
```

2.6. **A7: five semitones above the perfect interval reaches unison.** This is the acoustic constraint. It encodes the approximation $\log_2(3/2) \approx 7/12$: the perfect fifth is 7 semitones, so its inverse (the perfect fourth) is 5 semitones, and $7 + 5 = 12 = 0$ in $\mathbb{Z}/12\mathbb{Z}$.

Listing 7. `Axioms.lean`: A7

```
axiom five_above_perfect [NeZero m]
    (p : HarmonicInterval m) (hp : isPerfect p) : p + 5 = 0
```

2.7. **A8: a major/minor interval exists.** This rules out $m = 4$. In $\mathbb{Z}/4\mathbb{Z}$, the four elements are fully accounted for by unison, tritone, and the perfect pair; there is no room for a fourth class.

Listing 8. `Axioms.lean`: A8

```
axiom major_minor_exists [NeZero m]
    (p t : HarmonicInterval m) (hp : isPerfect p) (ht : IsTritone t) :
    ∃ i : HarmonicInterval m, i ≠ 0 ∧ ¬isSelfInverse i ∧ ¬isPerfect i
```

3. Derived Facts

Shared consequences. `tritone_eq` uses A2 (tritone uniqueness) to pin the tritone to a concrete value in any specific modulus, by supplying a witness and checking it satisfies the two defining properties. This eliminates repeated boilerplate in later proofs.

Listing 9. `Consequences.lean`: shared derived lemmas

```
theorem tritone_eq [NeZero m]
    (t : HarmonicInterval m) (ht : IsTritone t)
    (c : HarmonicInterval m) (hc0 : c ≠ 0) (hcsi : c + c = 0) : t = c :=
  tritone_unique t c ht { ne_zero := hc0, selfInverse := hcsi }

theorem selfInverse_eq_zero_or_tritone [NeZero m]
    (t i : HarmonicInterval m) (ht : IsTritone t) (hi : isSelfInverse i) :
    i = 0 ∨ i = t := by
  by_cases h : i = 0
  · exact Or.inl h
  · exact Or.inr (tritone_unique i t ⟨h, hi⟩ ht)

theorem perfect_not_self_inverse [NeZero m]
    (p : HarmonicInterval m) (hp : isPerfect p) : p ≠ p⁻¹ :=
  fun heq ⟹ perfect_not_SI p hp ((selfInverse_iff_inv_eq p).mpr heq.symm)


theorem perfect_pair_card [NeZero m]
    (p : HarmonicInterval m) (hp : isPerfect p) :
    ({p, p⁻¹} : Finset (HarmonicInterval m)).card = 2 :=
  Finset.card_pair (perfect_not_self_inverse p hp)
```

m is even. The tritone has additive order 2 (A1); the order of any element divides the group order $|\mathbb{Z}/m\mathbb{Z}| = m$; hence $2 \mid m$.

Listing 10. `TwelveTET.lean`: m is even

```
private lemma m_even [NeZero m]
    (t : HarmonicInterval m) (ht : IsTritone t) :
    2 ∣ m := by
  have hord : addOrderOf t = 2 := by
    haveI : Fact (Nat.Prime 2) := ⟨by decide⟩
    apply addOrderOf_eq_prime
    · rw [two_nsmul]; exact ht.selfInverse
    · exact ht.ne_zero
```

```
  have hdvd : addOrderOf t ∣ m := by
    have h : addOrderOf t ∣ Nat.card (ZMod m) := addOrderOf_dvd_natCard t
    rwa [Nat.card_eq_fintype_card, ZMod.card] at h  -- group order of ZMod m
      is m
  exact hord ▷ hdvd
```

$m \neq 2$**.** Every element of $\mathbb{Z}/2\mathbb{Z}$ is self-inverse, contradicting A4.

Listing 11. `TwelveTET.lean`: $m \neq 2$

```
private lemma m_ne_two [NeZero m]
    (p t : HarmonicInterval m) (hp : isPerfect p) (ht : IsTritone t) :
    m ≠ 2 := by
  intro hm; subst hm
  exact perfect_not_SI p hp (show p + p = 0 from by fin_cases p <;> decide)
```

$p = t + 1$**, derived from A6.** Since m is even and $m \neq 2$ we have $m \geq 4$, so $2 \neq 0$ in $\mathbb{Z}/m\mathbb{Z}$. Therefore $t + 1$ is not self-inverse (its double is $2 \neq 0$). By A6, $\mathrm{val}(p) \leq \mathrm{val}(t + 1) = \mathrm{val}(t) + 1$. Combined with $\mathrm{val}(t) < \mathrm{val}(p)$ from A6, this forces $p = t + 1$.

Listing 12. `TwelveTET.lean`: $p = t + 1$ derived from minimality

```
lemma perfect_above_tritone [NeZero m]
    (p t : HarmonicInterval m) (hp : isPerfect p) (ht : IsTritone t) :
    p = t + 1 := by
  obtain ⟨hgt, hmin⟩ := perfect_least_above_tritone p t hp ht
  have hm_pos : 0 < m := Nat.pos_of_ne_zero (NeZero.ne m)
  -- m ≥ 4: m is even and ≠ 2
  have hm4 : 4 ≤ m := by
    obtain ⟨k, hk⟩ := m_even t ht; have hn2 := m_ne_two p t hp ht; omega
  -- 2 ≠ 0 in ZMod m since m ≥ 4
  have h2ne : (2 : HarmonicInterval m) ≠ 0 := fun h ⟹
    absurd (Nat.le_of_dvd (by norm_num) ((CharP.cast_eq_zero_iff (ZMod m) m
    2).mp h))
           (by omega)
  -- t + 1 is not self-inverse: (t+1)+(t+1) = 2t+2 = 2 ≠ 0
  have ht1_nsi : ¬isSelfInverse (t + 1) := by
    intro h
    apply h2ne
    have : t + 1 + (t + 1) = 0 := h
    have : t + t = 0 := ht.selfInverse
    linear_combination ⟨t + 1 + (t + 1) = 0⟩ - ⟨t + t = 0⟩
  -- t + 1 ≠ 0
  have ht1_ne0 : t + 1 ≠ 0 := fun h ⟹ ht1_nsi (h ▷ unison_is_self_inverse
    )
  -- val(t + 1) = val t + 1: no wraparound since val t + 1 < m
  have hv1 : ZMod.val (1 : HarmonicInterval m) = 1 := ZMod.val_one'' (by omega
    )
  have hlt_m : ZMod.val t + ZMod.val (1 : HarmonicInterval m) < m := by
    rw [hv1]
    by_contra h
```

```
    push Not at h
    have heq : ZMod.val t + 1 = m := by have := ZMod.val_lt t; omega
    exact ht1_ne0 ((ZMod.val_eq_zero _).mp (by rw [ZMod.val_add, hv1, heq,
    Nat.mod_self]))
  have hval : ZMod.val (t + 1) = ZMod.val t + 1 := by
    rw [ZMod.val_add_of_lt hlt_m, hv1]
  -- By minimality: val p ≤ val(t+1) = val t + 1
  have hle : ZMod.val p ≤ ZMod.val (t + 1) :=
    hmin (t + 1) ht1_nsi (by omega)
  -- val p = val(t+1), so p = t+1
  exact ZMod.val_injective m (by omega)
```

$m \mid 12$**.** From $p = t + 1$ and A7: $t + 6 = (t + 1) + 5 = p + 5 = 0$. Then $12 = 2(t + 6) - (t + t) = 0$ in $\mathbb{Z}/m\mathbb{Z}$, so $m \mid 12$.

Listing 13. `TwelveTET.lean`: $m \mid 12$

```
private lemma tritone_plus_six [NeZero m]
    (p t : HarmonicInterval m) (hp : isPerfect p) (ht : IsTritone t) :
    t + 6 = 0 :=
  calc t + 6 = t + 1 + 5 := by ring
    _        = p + 5     := by rw [← perfect_above_tritone p t hp ht]
    _        = 0         := five_above_perfect p hp
private lemma twelve_eq_zero [NeZero m]
    (p t : HarmonicInterval m) (hp : isPerfect p) (ht : IsTritone t) :
    (12 : HarmonicInterval m) = 0 := by
  have hsi : t + t = 0 := ht.selfInverse
  linear_combination 2 * tritone_plus_six p t hp ht - hsi
private lemma m_dvd_twelve [NeZero m]
    (p t : HarmonicInterval m) (hp : isPerfect p) (ht : IsTritone t) :
    m ∣ 12 :=
  (CharP.cast_eq_zero_iff (ZMod m) m 12).mp (twelve_eq_zero p t hp ht)
```

$m \neq 6$ **and** $m \neq 4$**.** In $\mathbb{Z}/6\mathbb{Z}$: $t = 3$, $p = 4$, $p + 5 = 9 \neq 0$, contradicting A7. In $\mathbb{Z}/4\mathbb{Z}$: $t = 2$, $p = 3$, $p^{-1} = 1$; every element is in $\{0, t, p, p^{-1}\}$, contradicting A8.

Listing 14. `TwelveTET.lean`: eliminating $m = 6$ and $m = 4$

```
private lemma m_ne_six [NeZero m]
    (p t : HarmonicInterval m) (hp : isPerfect p) (ht : IsTritone t) :
    m ≠ 6 := by
  intro hm; subst hm
  have ht3 : t = 3 := tritone_eq t ht 3 (by decide) (by decide)
  have hp4 : p = 4 := by rw [perfect_above_tritone p t hp ht, ht3]; decide
  exact absurd (five_above_perfect p hp) (by simp only [hp4]; decide)
private lemma m_ne_four [NeZero m]
    (p t : HarmonicInterval m) (hp : isPerfect p) (ht : IsTritone t) :
    m ≠ 4 := by
  intro hm; subst hm
  obtain ⟨i, hine, hiSI, hiP⟩ := major_minor_exists p t hp ht
  have ht2 : t = 2 := tritone_eq t ht 2 (by decide) (by decide)
  have hp3 : p = 3 := by rw [perfect_above_tritone p t hp ht, ht2]; decide
```

```
  have hpi : p⁻¹ = 1 := by simp only [HI_inv_eq_neg, hp3]; decide
  fin_cases i <;> simp only at *
  · exact hine rfl                                          -- i = 0: unison
  · exact hiP (by have := perfect_inv_closed p hp; rwa [hpi] at this)  -- i = 1 = p⁻¹
  · exact hiSI (by simp only [isSelfInverse]; decide)          -- i = 2 = t
  · exact hiP (by rwa [hp3] at hp)                              -- i = 3 = p
```

The 12TET theorem: $m = 12$**.** Since $m \mid 12$, m is even, and $m \notin \{2, 4, 6\}$, the only remaining possibilities are $m \in \{12\}$ (with $m = 1$ excluded by `NeZero`). The `interval_cases` tactic dispatches the finite case split.

Listing 15. `TwelveTET.lean`: $m = 12$

```
theorem twelve_TET [NeZero m]
    (p t : HarmonicInterval m) (hp : isPerfect p) (ht : IsTritone t) :
    m = 12 := by
  have h1 := m_dvd_twelve p t hp ht
  have h2 := m_even t ht
  have h3 := m_ne_two p t hp ht
  have h4 := m_ne_four p t hp ht
  have h5 := m_ne_six p t hp ht
  have hm_pos : 0 < m := Nat.pos_of_ne_zero (NeZero.ne m)
  have hm_le : m ≤ 12 := Nat.le_of_dvd (by norm_num) h1
  interval_cases m <;> omega
```

Major/minor intervals as a least fixed point. The major/minor class is defined as the least fixed point of the step operator

$$\mathrm{stepMM}(p, t, s) = \big(s \cup \{j + 1 \mid j \in s \cup \{p, p^{-1}\}\} \cup \{j - 1 \mid j \in s \cup \{p, p^{-1}\}\}\big) \setminus \{0, t\}.$$

The lfp is the intersection of all prefixed points (sets s with $\mathrm{stepMM}(p, t, s) \subseteq s$).

Listing 16. `MajorMinor.lean`: definitions of selfInverseSet, stepMM, nonSIReachable, majorMinorIntervals

```
def selfInverseSet [NeZero m] (t : HarmonicInterval m) : Finset (HarmonicInterval m) :=
  {0, t}
def stepMM [NeZero m]
    (p t : HarmonicInterval m)
    (s : Finset (HarmonicInterval m)) : Finset (HarmonicInterval m) :=
  let frontier := s ∪ {p, p⁻¹}
  (s ∪ frontier.image (· + 1) ∪ frontier.image (· - 1)) \ selfInverseSet t
private theorem stepMM_monotone [NeZero m] (p t : HarmonicInterval m) :
    Monotone (stepMM p t) := by
  intro s s' hss' i hi
  simp only [stepMM, selfInverseSet, Finset.mem_sdiff, Finset.mem_union,
             Finset.mem_image, Finset.mem_insert, Finset.mem_singleton] at *
  obtain ⟨h, hni⟩ := hi
  refine ⟨?_, hni⟩
  rcases h with ((hs | hi1) | hi2)
```

```
  · exact Or.inl (Or.inl (hss' hs))
  · obtain ⟨j, hj, rfl⟩ := hi1
    rcases hj with hs | hp
    · exact Or.inl (Or.inr ⟨j, Or.inl (hss' hs), rfl⟩)
    · exact Or.inl (Or.inr ⟨j, Or.inr hp, rfl⟩)
  · obtain ⟨j, hj, rfl⟩ := hi2
    rcases hj with hs | hp
    · exact Or.inr ⟨j, Or.inl (hss' hs), rfl⟩
    · exact Or.inr ⟨j, Or.inr hp, rfl⟩
noncomputable def nonSIReachable [NeZero m]
    (p t : HarmonicInterval m) : Finset (HarmonicInterval m) :=
  Finset.univ.filter (fun i ⟹
    ∀ s : Finset (HarmonicInterval m), stepMM p t s ⊆ s → i ∈ s)

private theorem nonSIReachable_least [NeZero m]
    (p t : HarmonicInterval m) (s : Finset (HarmonicInterval m))
    (hs : stepMM p t s ⊆ s) : nonSIReachable p t ⊆ s := by
  intro i hi
  unfold nonSIReachable at hi
  rw [Finset.mem_filter] at hi
  exact hi.2 s hs
noncomputable def majorMinorIntervals [NeZero m]
    (p t : HarmonicInterval m) : Finset (HarmonicInterval m) :=
  nonSIReachable p t \ {p, p⁻¹}
```

The lfp equals the complement of the self-inverse set. The upper bound is immediate from monotonicity; the lower bound is proved by a 10-step walk in $\mathbb{Z}/12\mathbb{Z}$ starting from $p = 7$ and $p^{-1} = 5$.

Listing 17. `MajorMinor.lean`: upper bound

```
private theorem complement_SI_is_fixedPoint [NeZero m]
    (p t : HarmonicInterval m) :
    stepMM p t (Finset.univ \ selfInverseSet t) = Finset.univ \ selfInverseSet t
    := by
  ext i
  simp only [stepMM, selfInverseSet, Finset.mem_sdiff, Finset.mem_univ,
    true_and,
            Finset.mem_union, Finset.mem_image]
  constructor
  · rintro ⟨_, hni⟩; exact hni
  · intro hni
    -- i ∈ univ \ selfInverseSet t, so i ∈ s (the first component of the union
    )
    exact ⟨Or.inl (Or.inl hni), hni⟩
private theorem nonSIReachable_subset_complement [NeZero m]
    (p t : HarmonicInterval m) :
    nonSIReachable p t ⊆ Finset.univ \ selfInverseSet t :=
  nonSIReachable_least p t _ (complement_SI_is_fixedPoint p t).le
```

Listing 18. `MajorMinor.lean`: lower bound (10-step walk) and equality

```
private theorem prefixed_contains_complement [NeZero m]
    (p t : HarmonicInterval m)
    (hp : isPerfect p)
    (ht : IsTritone t)
    (s : Finset (HarmonicInterval m))
    (hpre : stepMM p t s ⊆ s) :
    Finset.univ \ selfInverseSet t ⊆ s := by
  have hm := twelve_TET p t hp ht
  subst hm
  have ht6 : t = 6 := tritone_eq t ht 6 (by decide) (by decide)
  have hp7 : p = 7 := by
    have := perfect_above_tritone p t hp ht; rw [ht6] at this; simpa using
    this
  have hpi5 : p⁻¹ = 5 := by
    simp only [HI_inv_eq_neg, hp7]; decide
  subst ht6; subst hp7
  -- {7, 5} = {p, p⁻¹};  {0, 6} = selfInverseSet t
  have step : ∀ j : ZMod 12,
      j ∈ s ∪ ({7, (5 : ZMod 12)} : Finset (ZMod 12)) →
      ∀ i : ZMod 12, (i = j + 1 ∨ i = j - 1) → i ∉ ({0, (6 : ZMod 12)} :
    Finset _) → i ∈ s := by
    intro j hj i hi hni
    apply hpre
    simp only [stepMM, selfInverseSet, Finset.mem_sdiff, Finset.mem_union,
               Finset.mem_image, hpi5]
    refine ⟨?_, hni⟩
    rcases hi with rfl | rfl
    · exact Or.inl (Or.inr ⟨j, Finset.mem_union.mp hj, rfl⟩)  -- i = j+1:
    frontier.image(·+1)
    · exact Or.inr ⟨j, Finset.mem_union.mp hj, rfl⟩            -- i = j-1:
    frontier.image(·-1)
  have fp7 : (7 : ZMod 12) ∈ s ∪ ({7, (5:ZMod 12)} : Finset _) := by simp
  have fp5 : (5 : ZMod 12) ∈ s ∪ ({7, (5:ZMod 12)} : Finset _) := by simp
  have h1 : (8 : ZMod 12) ∈ s :=
    step 7 fp7 8 (Or.inl (by decide)) (by decide)
  have h2 : (9 : ZMod 12) ∈ s :=
    step 8 (Finset.mem_union_left _ h1) 9 (Or.inl (by decide)) (by decide)
  have h3 : (10 : ZMod 12) ∈ s :=
    step 9 (Finset.mem_union_left _ h2) 10 (Or.inl (by decide)) (by decide)
  have h4 : (11 : ZMod 12) ∈ s :=
    step 10 (Finset.mem_union_left _ h3) 11 (Or.inl (by decide)) (by decide)
  have h5 : (4 : ZMod 12) ∈ s :=
    step 5 fp5 4 (Or.inr (by decide)) (by decide)
  have h6 : (3 : ZMod 12) ∈ s :=
    step 4 (Finset.mem_union_left _ h5) 3 (Or.inr (by decide)) (by decide)
  have h7 : (2 : ZMod 12) ∈ s :=
    step 3 (Finset.mem_union_left _ h6) 2 (Or.inr (by decide)) (by decide)
  have h8 : (1 : ZMod 12) ∈ s :=
```

```
    step 2 (Finset.mem_union_left _ h7) 1 (Or.inr (by decide)) (by decide)
  have h9 : (7 : ZMod 12) ∈ s :=
    step 8 (Finset.mem_union_left _ h1) 7 (Or.inr (by decide)) (by decide)
  have h10 : (5 : ZMod 12) ∈ s :=
    step 4 (Finset.mem_union_left _ h5) 5 (Or.inl (by decide)) (by decide)
  intro i hi
  simp only [selfInverseSet, Finset.mem_sdiff, Finset.mem_univ, true_and,
             Finset.mem_insert, Finset.mem_singleton] at hi
  push Not at hi
  obtain ⟨hi0, hi6⟩ := hi
  fin_cases i
  · exact absurd rfl hi0
  · exact h8
  · exact h7
  · exact h6
  · exact h5
  · exact h10
  · exact absurd rfl hi6
  · exact h9
  · exact h1
  · exact h2
  · exact h3
  · exact h4
    (p t : HarmonicInterval m)
    (hp : isPerfect p)
    (ht : IsTritone t) :
    Finset.univ \ selfInverseSet t ⊆ nonSIReachable p t := by
  intro i hi
  unfold nonSIReachable
  rw [Finset.mem_filter]
  exact ⟨Finset.mem_univ _, fun s hs ⟹ prefixed_contains_complement p t
    hp ht s hs hi⟩
    (p t : HarmonicInterval m)
    (hp : isPerfect p)
    (ht : IsTritone t) :
    nonSIReachable p t = Finset.univ \ selfInverseSet t :=
  Finset.Subset.antisymm
    (nonSIReachable_subset_complement p t)
    (complement_subset_nonSIReachable p t hp ht)
```

Cardinality.

Theorem 3.1. *The major/minor class has $m - 4$ elements; in* 12TET*, exactly 8.*

Listing 19. `Cardinality.lean`

```
theorem selfInverseSet_card [NeZero m]
    (t : HarmonicInterval m) (ht : IsTritone t) :
    (selfInverseSet t).card = 2 :=
  Finset.card_pair (Ne.symm ht.ne_zero)
```

```
private theorem nonSIReachable_card [NeZero m]
    (p t : HarmonicInterval m) (hp : isPerfect p) (ht : IsTritone t) :
    (nonSIReachable p t).card = m - 2 := by
  rw [nonSIReachable_eq_complement p t hp ht, Finset.card_sdiff]
  simp [Finset.card_univ, ZMod.card, selfInverseSet_card t ht]

private theorem perfect_pair_subset_nonSIReachable [NeZero m]
    (p t : HarmonicInterval m) (hp : isPerfect p) (ht : IsTritone t) :
    ({p, p⁻¹} : Finset (HarmonicInterval m)) ⊆ nonSIReachable p t := by
  rw [nonSIReachable_eq_complement p t hp ht]
  simp only [selfInverseSet, Finset.subset_sdiff, Finset.subset_univ, true_and,

             Finset.disjoint_left, Finset.mem_insert, Finset.mem_singleton]
  intro i hi
  rcases hi with rfl | rfl
  · simp only [not_or]
    exact ⟨perfect_nonzero _ hp, fun h ⟹ perfect_not_SI _ hp (h ▷ ht.
    selfInverse)⟩
  · simp only [not_or]
    exact ⟨by simp [HI_inv_eq_neg, neg_eq_zero, perfect_nonzero _ hp],
           fun h ⟹ perfect_not_SI _ (perfect_inv_closed _ hp) (h ▷ ht.
    selfInverse)⟩
theorem majorMinor_card [NeZero m]
    (p t : HarmonicInterval m) (hp : isPerfect p) (ht : IsTritone t) :
    (majorMinorIntervals p t).card = m - 4 := by
  simp only [majorMinorIntervals]
  have hsub := perfect_pair_subset_nonSIReachable p t hp ht
  have hcard_r := nonSIReachable_card p t hp ht
  have hcard_p := perfect_pair_card p hp
  rw [Finset.card_sdiff]
  have hint : {p, p⁻¹} ∩ nonSIReachable p t = {p, p⁻¹} :=
    Finset.inter_eq_left.mpr hsub
  rw [hint, hcard_r, hcard_p]; omega
theorem majorMinor_card_12 [NeZero m]
    (p t : HarmonicInterval m) (hp : isPerfect p) (ht : IsTritone t) :
    (majorMinorIntervals p t).card = 8 := by
  rw [majorMinor_card p t hp ht, twelve_TET p t hp ht]
```

4. The Partition Theorem

Theorem 4.1. *The four classes — $\{0\}$, $\{t\}$, $\{p, p^{-1}\}$, and the major/minor class — partition $\mathbb{Z}/m\mathbb{Z}$.*

Listing 20. `Partition.lean`

```
theorem interval_partition [NeZero m]
    (p t : HarmonicInterval m) (hp : isPerfect p) (ht : IsTritone t) :
    (Finset.univ : Finset (HarmonicInterval m)) =
      selfInverseSet t ∪ {p, p⁻¹} ∪ majorMinorIntervals p t := by
```

```
  simp only [majorMinorIntervals, selfInverseSet ]
  rw [nonSIReachable_eq_complement p t hp ht]
  ext i
  simp only [Finset.mem_univ, Finset.mem_union, Finset.mem_insert, Finset.
    mem_singleton,
             Finset.mem_sdiff, true_iff]
  by_cases h0  : i = 0;    · exact Or.inl (Or.inl (Or.inl h0))
  by_cases ht' : i = t;    · exact Or.inl (Or.inl (Or.inr ht'))
  by_cases hp' : i = p;    · exact Or.inl (Or.inr (Or.inl hp'))
  by_cases hpi : i = p⁻¹; · exact Or.inl (Or.inr (Or.inr hpi))
  · right
    refine ⟨⟨trivial, ?_⟩, ?_⟩
    · simp only [selfInverseSet, Finset.mem_insert, Finset.mem_singleton,
    not_or]
      exact ⟨h0, ht'⟩
    · simp only [not_or]
      exact ⟨hp', by rwa [HI_inv_eq_neg]⟩
```

5. Risks to Validity

Axiom consistency. Since isPerfect is opaque, the axioms cannot be proved as theorems. We address this by bundling the eight axiom conditions into a structure `HarmonicModel`, parameterised over an explicit predicate `perf` that stands in for isPerfect. A term of this type for a specific modulus is a consistency certificate.

Listing 21. `Model.lean`: axiom bundle and consistency certificate

```
structure HarmonicModel (m : ℕ) [NeZero m] where
  t    : HarmonicInterval m
  p    : HarmonicInterval m
  perf : HarmonicInterval m → Prop
  A1   : IsTritone t
  /-- A2: the tritone is unique. -/
  A2   : ∀ s : HarmonicInterval m, IsTritone s → s = t
  /-- A3: a perfect interval exists. -/
  A3   : perf p
  /-- A4: perfect intervals are not self-inverse. -/
  A4   : ∀ q, perf q → ¬isSelfInverse q
  /-- A5: the perfect class is exactly {p, p⁻¹}. -/
  A5   : ∀ q, perf q ↔ q = p ∨ q = p⁻¹
  /-- A6: p is the least non-self-inverse interval above t. -/
  A6   : ZMod.val t < ZMod.val p ∧
         ∀ i : HarmonicInterval m, ¬isSelfInverse i →
           ZMod.val t < ZMod.val i → ZMod.val p ≤ ZMod.val i
  /-- A7: five semitones above p reaches unison. -/
  A7   : p + 5 = 0
  /-- A8: a non-unison non-self-inverse non-perfect interval exists. -/
  A8   : ∃ i : HarmonicInterval m, i ≠ 0 ∧ ¬isSelfInverse i ∧ ¬perf i

/-- The standard model: `ZMod 12`, `t = 6`, `p = 7`, `perf = {7, 5}`.
```

```
All conditions are verified by ‘decide‘ or ‘fin_cases‘ + ‘decide‘. -/
def zmod12Model : HarmonicModel 12 where
  t    := 6
  p    := 7
  perf := fun i ⟹ i = 7 ∨ i = 5
  A1   := ⟨by decide, by decide⟩
  A2   := by
    intro s ⟨hne, hsi⟩
    fin_cases s <;> simp_all [isSelfInverse]
  A3   := Or.inl rfl
  A4   := by
    intro q hq
    rcases hq with rfl | rfl <;> simp only [isSelfInverse] <;> decide
  A5   := by
    intro q
    simp only [HI_inv_eq_neg]
    fin_cases q <;> decide
  A6   := ⟨by decide, by
    intro i hsi hgt
    fin_cases i <;> revert hsi hgt <;> simp only [isSelfInverse] <;> decide⟩
  A7   := by decide
  A8   := ⟨1, by decide, by simp only [isSelfInverse]; decide, by decide⟩
```

The model is $\mathbb{Z}/12\mathbb{Z}$ with $t = 6$, $p = 7$, and $\mathrm{perf}\, i \Leftrightarrow i = 7 \vee i = 5$. All eight conditions are discharged computationally by `decide` or `fin_cases` followed by `decide`.

The numeral 5 in A7. Axiom A7 ($p + 5 = 0$) introduces the literal 5. This is the irreducible acoustic content of the proof: the perfect fifth is approximated by 7/12 of an octave, a fact about physics that cannot be derived from qualitative musical structure alone. Any reformulation of A7 that avoids the numeral 5 must implicitly encode the same information.

The minimality axiom A6. The minimality formulation of A6 avoids the literal 1 of the original $p = t + 1$, but the derived lemma `perfect_above_tritone` still concludes $p = t + 1$. The improvement is that the *axiom* is now qualitative; the numeral emerges from the proof rather than being assumed.

The opaque predicate isPerfect. Declaring isPerfect opaque is load-bearing: it prevents Lean from unfolding a hypothetical explicit definition during proof search, which could produce spurious proofs. The trade-off is that isPerfect cannot be computationally evaluated.

6. Further Work

- **Formalise a model.** Construct $\mathbb{Z}/12\mathbb{Z}$ explicitly with $t = 6$, $p = 7$, and verify all eight axioms, giving a consistency proof.
- **Eliminate A7.** Replace $p + 5 = 0$ with an axiom expressing acoustic nearness (best rational approximation of $\log_2(3/2)$) and derive $p + 5 = 0$ as a theorem. This requires bringing in real analysis or continued-fraction arithmetic.

- **Diatonic structure.** Characterise the major scale as a specific 7-element subset of the major/minor class and derive its interval structure (tone–semitone pattern) from the axioms.
- **Voice-leading.** Formalise the claim that the major/minor class is closed under minimal voice-leading displacement (± 1 step), connecting the lfp construction to the musical concept.

Appendix A. Lean 4 Proof Tactics

The following tactics appear in the proof. Each entry gives the tactic name as it appears in the source, followed by a brief plain-English description of what it does.

`apply`: Unify the current goal with the conclusion of a given lemma or hypothesis, leaving the lemma's hypotheses as new sub-goals.

`by_cases`: Split the proof into two branches by introducing a hypothesis $h : P$ in one branch and $h : \neg P$ in the other.

`by_contra`: Replace the current goal P with the assumption $\neg P$ and derive a contradiction.

`calc`: Chain a sequence of equalities or inequalities, where each step is justified by a tactic or term.

`constructor`: Split a conjunction or existential goal into its components, producing one sub-goal per component.

`decide`: Discharge a decidable proposition by computation (kernel-level evaluation).

`exact`: Close the current goal by providing a term whose type matches it exactly.

`ext`: Reduce an equality of functions, sets, or finsets to pointwise equality, introducing a universally quantified variable.

`fin_cases`: Enumerate all cases for a variable ranging over a finite type, reducing the goal to one instance per element.

`have`: Introduce a local hypothesis by stating its type and providing a proof; the main goal is then proved with the new hypothesis in context.

`haveI`: Like `have`, but inserts the result as a local instance so that typeclass search can use it immediately.

`interval_cases`: When a variable is constrained to a finite range of integers by hypotheses in the context, enumerate each integer in that range as a separate case.

`intro`: Move a universally quantified variable or an implication hypothesis from the goal into the local context.

`linear_combination`: Close a goal that is a linear combination of hypotheses by supplying the coefficients; the remainder is discharged by `ring`.

`norm_num`: Evaluate and close goals involving numerical expressions (arithmetic, divisibility, inequalities on literals).

`nth_rw`: Like `rw`, but rewrites only the n-th occurrence of the pattern rather than all occurrences.

`obtain`: Destruct a hypothesis of product, existential, or disjunctive type, binding the components to named variables.

`omega`: Decide linear arithmetic goals over the integers or natural numbers.

`push Not`: Push negations inward through logical connectives and quantifiers, converting e.g. $\neg(\forall x, P)$ to $\exists x, \neg P$.

`rcases`: Recursively destruct a hypothesis using a pattern, binding sub-terms to named variables in one step.

`refine`: Like `exact`, but allows placeholders (`?_`) that become new sub-goals, supporting partial term construction.

`rfl`: Close a goal of the form $a = a$ by reflexivity.

`ring`: Close a goal that is an identity in a commutative (semi)ring by normalisation.

`rintro`: Combine `intro` and `rcases`: introduce a hypothesis and immediately destruct it using a pattern.

`rw`: Rewrite the goal (or a hypothesis) by substituting equals for equals, using an equation or iff.

`rwa`: Like `rw`, but closes the goal with `assumption` after the rewrite.

`simp`: Simplify the goal (or a hypothesis) by repeatedly applying a configurable set of rewrite lemmas and decision procedures.

`simp_all`: Apply `simp` to the goal and all hypotheses simultaneously, using each simplified hypothesis to further simplify the others.

`subst`: Given a hypothesis $x = e$ (or $e = x$) where x is a local variable, substitute e for x everywhere and remove the variable.

`unfold`: Unfold a definition by name in the goal, replacing the defined identifier with its body.

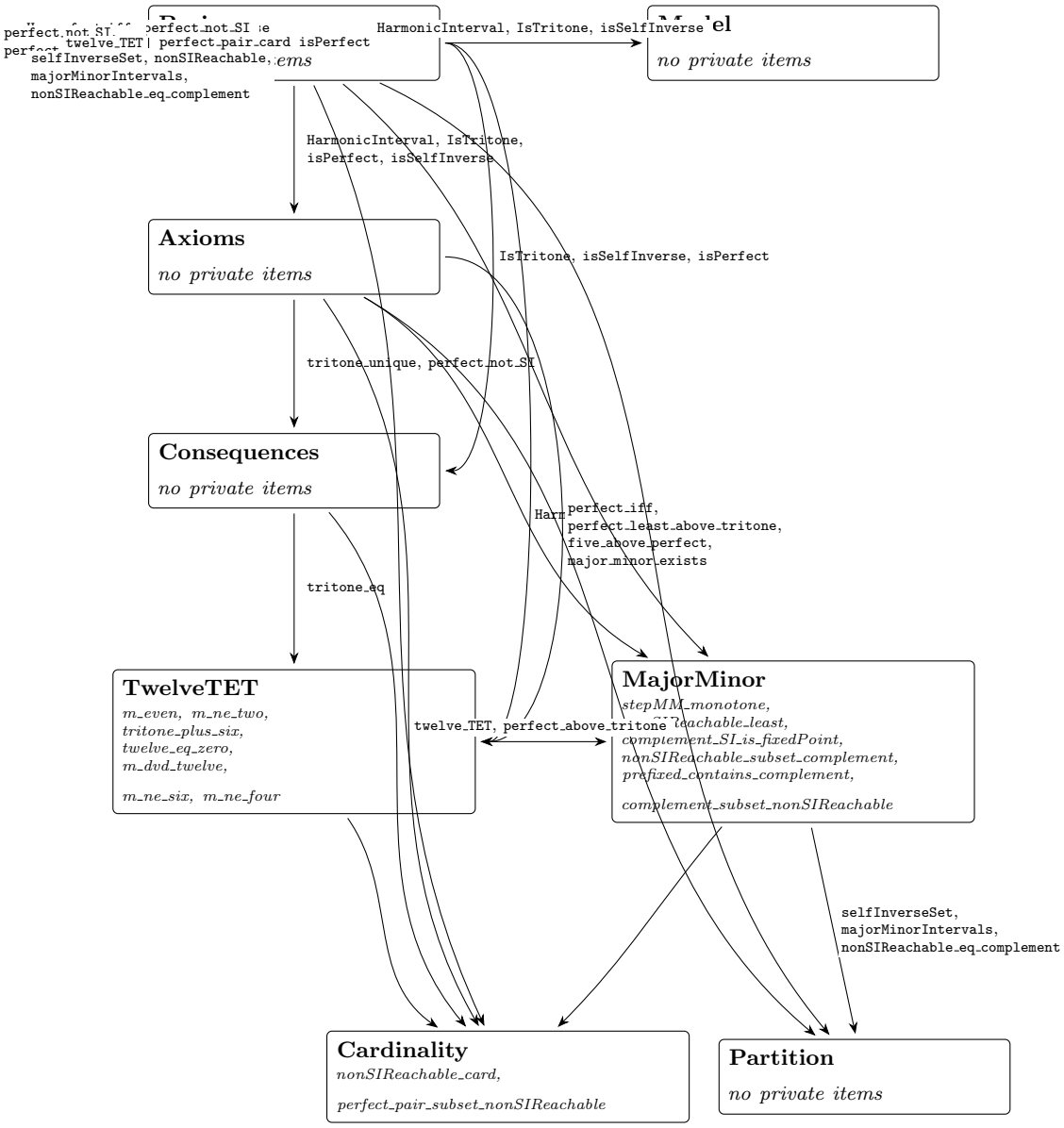

FIGURE 1. Lean module architecture. Private declarations are listed in italics inside each rectangle. Each arrow is labelled with the public names from the source module that are used in the destination.